Kubernetes Taints & Tolerations

We'll do **6 practicals**, from basic to real-world:

Lab	Practical	What you learn
1	Apply a NoSchedule taint	Block normal Pods
2	Add a toleration	Allow Pod onto tainted node
3	Toleration + NodeSelector	Actually target a specific node
4	PreferNoSchedule	Understand soft taint
5	NoExecute	Evict existing Pods
6	tolerationSeconds	Delay eviction

Important: These labs work best with at least **2 worker nodes**. If you have only one node, I'll show a single-node alternative where possible.

Lab 1 — NoSchedule Taint

Objective

Create a taint on a node and prove that a normal Pod cannot be scheduled there.

Step 1 — Check your nodes

kubectl get nodes

Example:

NAME STATUS ROLES

node-1 Ready <none>

node-2 Ready <none>

Choose node-1 for this lab.

Step 2 — Apply taint

kubectl taint nodes node-1 workload=gpu:NoSchedule

Expected:

node/node-1 tainted

The taint is:

workload=gpu:NoSchedule

Step 3 — Verify

kubectl describe node node-1

Look near the bottom for:

Taints:

workload=gpu:NoSchedule

You can also use:

kubectl get node node-1 -o jsonpath='{.spec.taints}'

Step 4 — Create a normal Pod

Create:

01-normal-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: normal-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.27
```

Apply:

kubectl apply -f 01-normal-pod.yaml

Check:

kubectl get pod normal-pod -o wide

If the cluster has another suitable node, the Pod may schedule there.

To see scheduling details:

kubectl describe pod normal-pod

Look at:

Events:

Important observation

The Pod isn't necessarily "forbidden from the entire cluster."

It is forbidden from **nodes whose taints it does not tolerate**.

Lab 2 — Add a Toleration

Now we'll allow our Pod to tolerate:

---

workload=gpu:NoSchedule

Create:

02-toleration-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: toleration-pod
spec:

 tolerations:
   - key: workload
     operator: Equal
     value: gpu
     effect: NoSchedule


 containers:
   - name: nginx
     image: nginx:1.27
```

Apply:

```
kubectl apply -f 02-toleration-pod.yaml
```

Check:

```
kubectl get pod toleration-pod -o wide
```

Very Important Observation

We added:

```
tolerations:
  - key: workload
    operator: Equal
    value: gpu
    effect: NoSchedule
```

This means:

```
Pod
 |
| "I tolerate"
↓
workload=gpu:NoSchedule
```

Therefore:

```
Node
workload=gpu:NoSchedule
      ↑
      |
      Pod tolerates it
```

But remember:

Toleration allows the Pod; it doesn't force the Pod onto node-1.

Lab 3 — Toleration + NodeSelector

This is the most important practical.

We want:

"Only GPU workload should run on the GPU node."

We therefore need:

Taint

+

Toleration

+

NodeSelector

Step 1 — Label node-1

kubectl label node node-1 workload=gpu

Verify:

kubectl get nodes --show-labels

Or:

kubectl get node node-1 --show-labels

You should see:

workload=gpu

Step 2 — Create Pod

03-gpu-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:

  nodeSelector:
    workload: gpu

  tolerations:
    - key: workload
      operator: Equal
```

```
    value: gpu
    effect: NoSchedule

  containers:
    - name: nginx
      image: nginx:1.27
```

Apply:

```
kubectl apply -f 03-gpu-pod.yaml
```

Check:

```
kubectl get pod gpu-pod -o wide
```

You should see it scheduled on:

```
node-1
```

Understand What Happened

We used two mechanisms.

Toleration

```
tolerations:
  - key: workload
    operator: Equal
    value: gpu
    effect: NoSchedule
```

Means:

"I'm allowed to use a GPU node."

NodeSelector

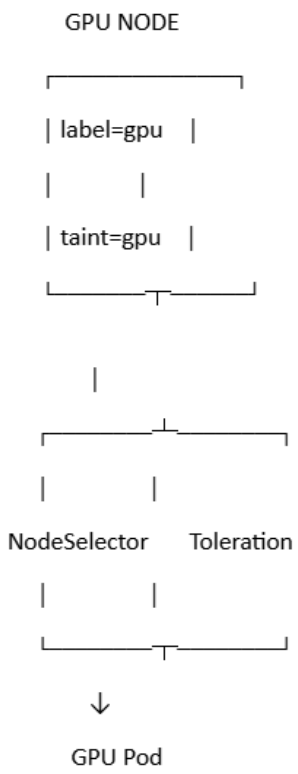
```
nodeSelector:
```

workload: gpu

Means:

"I specifically want a node labeled workload=gpu."

Together:



This is a **very common production pattern**.

Lab 4 — PreferNoSchedule

Now let's understand the soft taint.

First remove the old taint:

kubectl taint nodes node-1 workload=gpu:NoSchedule-

Now add:

---

kubectl taint nodes node-1 workload=gpu:PreferNoSchedule

Verify:

kubectl describe node node-1

You should see:

Taints:

  workload=gpu:PreferNoSchedule

Create normal Pod

04-prefer-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: prefer-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.27
```

Apply:

kubectl apply -f 04-prefer-pod.yaml

Check:

kubectl get pod prefer-pod -o wide

What should you understand?

Unlike:

NoSchedule

PreferNoSchedule is not a hard block.

Think:

NoSchedule

↓

"Don't put it here."

PreferNoSchedule

↓

"Try not to put it here."

The scheduler can still place the Pod there if necessary.

Lab 5 — NoExecute

This is one of the most important labs.

NoExecute can affect **existing Pods**.

Step 1 — Remove previous taint

kubectl taint nodes node-1 workload=gpu:PreferNoSchedule-

Step 2 — Create a Pod specifically on node-1

We'll use nodeSelector temporarily.

05-noexecute-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: noexecute-pod
spec:
  nodeSelector:
    workload: gpu

  containers:
    - name: nginx
      image: nginx:1.27
```

Apply:

```
kubectl apply -f 05-noexecute-pod.yaml
```

Check:

```
kubectl get pod noexecute-pod -o wide
```

It should be running on node-1.

Step 3 — Apply NoExecute

Now run:

```
kubectl taint nodes node-1 maintenance=true:NoExecute
```

Immediately check:

```
kubectl get pods -o wide
```

The Pod that doesn't tolerate:

maintenance=true:NoExecute

can be evicted.

Lab 6 — tolerationSeconds

Now we'll make a Pod tolerate a NoExecute taint temporarily.

First remove the taint:

---

kubectl taint nodes node-1 maintenance=true:NoExecute-

Create Pod

06-toleration-seconds.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: temporary-toleration
spec:

  nodeSelector:
    workload: gpu

  tolerations:
    - key: maintenance
      operator: Equal
      value: true
      effect: NoExecute
      tolerationSeconds: 30
  containers:
    - name: nginx
      image: nginx:1.27
```

Apply:

kubectl apply -f 06-toleration-seconds.yaml

Verify:

kubectl get pod temporary-toleration -o wide

Apply NoExecute

kubectl taint nodes node-1 maintenance=true:NoExecute

Immediately watch:

kubectl get pod temporary-toleration -w

The Pod can remain temporarily.

After approximately:

30 seconds

it can be evicted because:

tolerationSeconds: 30

Lab 7 — Exists Operator

Now let's practice another important toleration.

Instead of:

operator: Equal

we can use:

operator: Exists

Example:

```
tolerations:
  - key: workload
    operator: Exists
    effect: NoSchedule
```

This means:

"I tolerate a workload taint regardless of its value."

So this can match:

workload=gpu:NoSchedule

and:

workload=database:NoSchedule

because the key is:

workload

Lab 8 — Complete Real-World Example

Let's create a realistic dedicated-node setup.

Imagine:

Node-1

---

label:

---

workload=database

taint:

workload=database:NoSchedule

Only database Pods should run there.

Step 1 — Label

kubectl label node node-1 workload=database --overwrite

Step 2 — Taint

kubectl taint node node-1 workload=database:NoSchedule

Step 3 — Database Pod

database-pod.yaml

apiVersion: v1

kind: Pod

metadata:

name: database-pod

```
spec:
  nodeSelector:
    workload: database

  tolerations:
    - key: workload
      operator: Equal
      value: database
      effect: NoSchedule

  containers:
    - name: postgres
      image: postgres:16

      env:
        - name: POSTGRES_PASSWORD
          value: "password"
```

Apply:

```
kubectl apply -f database-pod.yaml
```

Check:

```
kubectl get pod database-pod -o wide
```

Verify Everything

Check node taints

```
kubectl get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
```

Or:

```
kubectl describe node node-1
```

Check Pod tolerations

```
kubectl get pod database-pod -o yaml
```

Look for:

```
tolerations:
```

Check where Pod is running

kubectl get pods -o wide

Check scheduling events

kubectl describe pod database-pod

Especially:

Events:

This is extremely useful when teaching students how to troubleshoot scheduling.

Cleanup

Remove taints:

kubectl taint nodes node-1 workload=gpu:NoSchedule-

kubectl taint nodes node-1 workload=gpu:PreferNoSchedule-

---

kubectl taint nodes node-1 maintenance=true:NoExecute-

kubectl taint nodes node-1 workload=database:NoSchedule-

Remove labels:

kubectl label node node-1 workload-

Delete lab Pods:

kubectl delete pod normal-pod toleration-pod gpu-pod prefer-pod noexecute-pod temporary-toleration database-pod

Interview summary

Concept	**Meaning**
Taint	Repels Pods from a Node
Toleration	Allows a Pod to tolerate a taint
NoSchedule	Don't schedule new non-tolerating Pods
PreferNoSchedule	Prefer not to schedule
NoExecute	Can evict existing non-tolerating Pods
tolerationSeconds	Tolerate NoExecute temporarily
Equal	Key/value must match
Exists	Key must exist
NodeSelector	Selects nodes based on labels
Toleration alone	**Does not guarantee placement**